



SISTEMAS DE TIEMPO REAL

PRÁCTICA 2



12 DE ENERO DE 2025

POR ARNAU FAURA CIRÉ Y LUIS MANUEL OCAMPO ÁVILA

Tabla de Contenidos

Introducción	3
EJERCICIO 1 – Hilos	4
Especificación	4
Diseño	4
Implementación	4
Ejecución	6
EJERCICIO 2 – POSIX <i>Timers</i>	7
Especificación	7
Diseño	7
Implementación	7
Ejecución	8
EJERCICIO 3 – Colas de Mensajes	9
Especificación	9
Diseño	9
Implementación	9
Ejecución	10
EJERCICIO 4 – Memoria Compartida	11
Especificación	11
Diseño	11
Implementación	11
Ejecución	12
EJERCICIO 5 – Señales de Tiempo Real	13
Especificación	13
Diseño	13
Implementación	13
Ejecución	14
EJERCICIO 6 – Inversión de Prioridades	15
Especificación	15
Diseño	15
Implementación	15

Ejecución.....	16
----------------	----

Tabla de Ilustraciones

Ilustración 1 - Ejecución del Ejercicio 1	6
Ilustración 2 - Ejecución del Ejercicio 2	8
Ilustración 3 - Ejecución del Ejercicio 3	10
Ilustración 4 - Ejecución del Ejercicio 4	12
Ilustración 5 - Ejecución del Ejercicio 5	14
Ilustración 6 - Ejecución del Ejercicio 6 (Con Inversión de Prioridades)	16
Ilustración 7 - Ejecución del Ejercicio 6 (Sin Inversión de Prioridades)	16

Introducción

Los sistemas de tiempo real constituyen una parte fundamental de numerosos entornos críticos, donde la corrección del sistema no solo depende del resultado lógico de las operaciones, sino también del cumplimiento estricto de restricciones temporales. En este contexto, el estándar POSIX proporciona un conjunto de mecanismos que permiten desarrollar aplicaciones capaces de gestionar concurrencia, temporización y comunicación entre procesos de forma determinista y eficiente.

La presente práctica tiene como objetivo profundizar en el uso de las principales herramientas que ofrece POSIX para la programación de sistemas de tiempo real. A lo largo de los distintos ejercicios se trabajan conceptos clave como la creación y planificación de hilos con políticas de tiempo real, el uso de temporizadores POSIX, la comunicación mediante colas de mensajes, la compartición de memoria entre procesos, el manejo de señales de tiempo real y la problemática de la inversión de prioridades.

Mediante la implementación práctica y la observación del comportamiento del sistema bajo diferentes configuraciones, esta práctica permite comprender mejor el impacto de las prioridades, las políticas de planificación y los mecanismos de sincronización en el rendimiento y la predictibilidad de un sistema de tiempo real. El documento recoge las soluciones implementadas, así como el análisis y las conclusiones extraídas de los experimentos realizados.

EJERCICIO 1 – Hilos

Especificación

- 1.1. Escriba un programa que cree múltiples hilos de tiempo real utilizando la función `pthread_create()`.
- 1.2. Asigne diferentes políticas de planificación (*scheduling policies*), por ejemplo: `SCHED_FIFO`, `SCHED_RR`, y diferentes prioridades a cada uno de los hilos.
- 1.3. Observe el comportamiento de los hilos y documéntelo.

Diseño

Antes de comentar el diseño del programa, es conveniente aclarar el funcionamiento de las políticas de planificación con prioridades en el estándar POSIX. Después de buscar información y hacer algunas pruebas se ha simplificado el funcionamiento en las siguientes directrices:

1. El hilo con un valor de prioridad más alto se ejecutará antes que uno con un valor más bajo, independientemente de la política de planificación asociada a cada uno.
2. Los hilos que tengan el mismo valor de prioridad se planificarán entre ellos según la política de planificación de cada uno.
Por ejemplo, un hilo Round Robin después de ejecutar durante un número de *quantums*, cedería la CPU a un hilo FIFO de igual prioridad y este usará la CPU hasta que termine su ejecución.

Teniendo esto claro, pasemos a ver la estructura del programa propuesto. Se ha diseñado un programa que crea 4 hilos con distintas políticas de planificación y prioridades. Cada hilo ejecuta un bucle de 8 iteraciones donde cada una mostrará por pantalla un mensaje indicando el identificador de hilo y el número de iteración.

Implementación

A continuación, se muestra una tabla con la configuración de los hilos:

Hilo	Política de Planificación	Prioridad
Hilo 1	SCHED_RR	10
Hilo 2	SCHED_RR	10
Hilo 3	SCHED_FIFO	20
Hilo 4	SCHED_FIFO	20

En el bucle de muestra de mensajes de cada hilo se ha añadido el siguiente bucle en el que simplemente se hace una multiplicación muchas veces, con el objetivo de causar un retardo en la ejecución del hilo sin que este libere la CPU. Si esto sucediese el esquema de hilos podría comportarse de manera indeterminada.

```
// volatile int temp;

for ( i = 0; i < 8; i++ ) {
    for ( j = 0; j < 100000000; j++ ) {
        temp = j * j;
    }
    printf("Hilo %d : %d\n", data->hilo, i);
}
```

Teniendo una configuración de hilos como la anterior, los resultados esperados deberían ser parecidos a lo siguiente. Los hilos 3 y 4, al tener mayor prioridad se ejecutarían antes que los hilos 1 y 2, o los interrumpirían en caso de que se hayan creado antes. A partir de aquí, teniendo en cuenta que los hilos 3 y 4 se planifican con SCHED_FIFO, el primero de ambos que haya obtenido la CPU la usará hasta terminar y luego el siguiente hará lo mismo. Cuando los hilos 3 y 4 hayan acabado, los hilos 1 y 2 se repartirán la CPU hasta que acaben, debido a que fueron planificados mediante SCHED_RR.

Ejecución

Con el objetivo de lograr la **predictibilidad**, una característica importante en sistemas de tiempo real, se ha ajustado el número de CPUs a 1 en la máquina virtual donde se ha realizado este ejercicio. De esta forma se garantiza que la planificación de hilos funcione según lo esperado.

También hay que comentar que para usar las planificaciones SCHED_FIFO y SCHED_RR el programa debe ejecutarse con permisos de superusuario.

Finalmente se muestra la ejecución del programa.

```
milax@casa:~/STR/str-p2/src/ex1$ sudo ./ex1.out
Hilo 4: SCHED = 1, PRIOR = 20
Hilo 4 : 0
Hilo 4 : 1
Hilo 4 : 2
Hilo 4 : 3
Hilo 4 : 4
Hilo 3: SCHED = 1, PRIOR = 20
Hilo 2: SCHED = 2, PRIOR = 10
Hilo 1: SCHED = 2, PRIOR = 10
Hilo 4 : 5
Hilo 4 : 6
Hilo 4 : 7
Hilo 3 : 0
Hilo 3 : 1
Hilo 3 : 2
Hilo 3 : 3
Hilo 3 : 4
Hilo 3 : 5
Hilo 3 : 6
Hilo 3 : 7
Hilo 2 : 0
Hilo 1 : 0
Hilo 2 : 1
Hilo 1 : 1
Hilo 2 : 2
Hilo 1 : 2
Hilo 2 : 3
Hilo 1 : 3
Hilo 2 : 4
Hilo 1 : 4
Hilo 2 : 5
Hilo 1 : 5
Hilo 2 : 6
Hilo 1 : 6
Hilo 2 : 7
Hilo 1 : 7
```

Ilustración 1 - Ejecución del Ejercicio 1

Se aprecia en la imagen que el comportamiento real es similar al esperado. Los hilos se crean en orden descendiente. El primero en obtener la CPU es el hilo 4. Aunque se crean los demás, el hilo 4 acaba su ejecución para darle paso al hilo 3. Cuando este acaba, los siguientes con menor prioridad se reparten la CPU hasta terminar.

EJERCICIO 2 – POSIX *Timers*

Especificación

- 2.1. Escribir un programa que utilice un *timer* de POSIX.
- 2.2. Configurándolo para enviar una señal UNIX de tipo SIGALRM en intervalos regulares
- 2.3. Y cada vez que se reciba esta señal se ejecute la función `my_handler(int signum)`.

Diseño

Asociación de Señales:

El programa debe registrar primero una función específica (*Handler*) que se llamará al recibir la señal SIGALRM, reemplazando así el comportamiento predeterminado del sistema (que suele ser la finalización del programa).

Configuración del Temporizador (Structs):

- **Notificación** (`sigevent`): Define cómo el temporizador notifica al proceso. Lo diseñamos para generar un SIGALRM.
- **Temporización** (`itimerspec`): Define cuándo se activa el temporizador. Requiere dos valores distintos: Caducidad Inicial: Tiempo de espera antes de la primera señal. Intervalo: Periodo entre todas las señales subsiguientes.

Flujo de Control:

- **Inicialización:** Configurar estructuras -> Crear Temporizador -> Armar Temporizador.
- **Bucle Inactivo:** El hilo principal entra en un bucle infinito mediante `pause()`. Esto suspende el proceso (sin usar CPU) hasta que una señal lo reactiva.
- **Contexto de Interrupción:** Cuando el temporizador se activa, el SO pausa el bucle principal, ejecuta el *Handler* y reanuda el bucle.

Implementación

Estructuras:

- **struct sigaction:** Se utiliza para vincular de forma segura SIGALRM a la función personalizada `my_handler`. Se usa `sigemptyset` para garantizar que ninguna otra señal bloquee el controlador.
- **struct sigevent:** Se configura con `SIGEV_SIGNAL` para garantizar que la expiración genere una señal en lugar de un hilo.

- **struct itimerspec:**
 - `it_value.tv_sec = 1`: El temporizador comienza 1 segundo después de llamar a `timer_settime`.
 - `it_interval.tv_sec = 2`: Después de la primera ejecución, se repite cada 2 segundos.

Funciones:

- **timer_create(CLOCK_REALTIME, ...)**: Inicializa un nuevo temporizador por proceso utilizando el reloj de tiempo real del sistema.
- **timer_settime(...)**: Activa el temporizador. Sin esta llamada, el temporizador existe, pero está inactivo.
- **pause()**: Esto es crucial para la eficiencia. En lugar de un bucle `while(true)` que gira y consume el 100% de la CPU, `pause()` suspende el proceso por completo hasta que llega una señal.

Handler de señales:

Acepta el número de señal como argumento.

Utiliza un contador global `i` para registrar el número de ejecuciones.

Utiliza `printf` para mostrar el estado en la salida estándar.

Ejecución

El output de la consola es el siguiente y se puede ver que se ejecuta cada 2 segundos el *handler* del *timer*, a excepción de la primera vez, que solo tarda 1 segundo:

```
milax@casa:~/STR/str-p2/src/ex2$ make run
./ex2.out
Timer created successfully.
Timer started... Waiting for signals
Caught signal 14 - Timer expired! Execution number 1
Caught signal 14 - Timer expired! Execution number 2
Caught signal 14 - Timer expired! Execution number 3
Caught signal 14 - Timer expired! Execution number 4
Caught signal 14 - Timer expired! Execution number 5
Caught signal 14 - Timer expired! Execution number 6
Caught signal 14 - Timer expired! Execution number 7
```

Ilustración 2 - Ejecución del Ejercicio 2

EJERCICIO 3 – Colas de Mensajes

Especificación

- 3.1. Escriba un programa productor–consumidor utilizando colas POSIX (`mq_open()`, `mq_send()`, `mq_receive()`). **Nota:** Asegúrese de que el productor y el consumidor funcionen en hilos diferentes.
- 3.2. Experimente con diferentes prioridades de los mensajes y observe el impacto en el procesamiento de los mensajes por parte del consumidor.
- 3.3. Documente la aplicación y el comportamiento observado.

Diseño

Para hacer este ejercicio se ha propuesto una estructura de productor y consumidor. Como se especifica que programa use hilos para cada rol, todo el código fuente se ha escrito en el mismo archivo.

El diseño y funcionamiento del programa es el siguiente. El programa principal crea e inicializa la cola de mensajes. Aunque el formato de esta es flexible, comúnmente se usa para intercambiar mensajes de texto. Eso es lo que haremos.

Posteriormente se creará un hilo productor que insertará mensajes en la cola con distintas prioridades. Cuando este acabe, se iniciará un hilo consumidor que obtendrá los mensajes de la cola. Ambos hilo imprimirán por pantalla información para sacar conclusiones.

Implementación

- **Hilo Productor:** Este hilo realiza un bucle que ejecuta `mq_send()` hasta que se llene la cola. Los mensajes que envía tienen el formato "MSG i", donde i es la iteración del bucle numerada a partir de 1. Cada mensaje se le asocia una prioridad igual a 0 o 1 en función de si i es par o impar, respectivamente.
- **Hilo Consumidor:** Este hilo realiza un bucle que ejecuta `mq_receive()` tantas veces como mensajes haya enviado el hilo productor.

El hilo consumidor se ejecuta cuando el productor ha acabado. El formato en que los hilos muestran la información es el siguiente:

```
[PROD | CONS] -> | <- (mensaje, prioridad)
```

Ejecución

Se muestra a continuación una captura de pantalla de la ejecución del programa.

```
milax@casa:~/STR/str-p2/src/ex3$ ./ex3.out
[PROD] -> ( MSG 1, 1)
[PROD] -> ( MSG 2, 0)
[PROD] -> ( MSG 3, 1)
[PROD] -> ( MSG 4, 0)
[PROD] -> ( MSG 5, 1)
[PROD] -> ( MSG 6, 0)
[PROD] -> ( MSG 7, 1)
[PROD] -> ( MSG 8, 0)
[PROD] -> ( MSG 9, 1)
[PROD] -> ( MSG 10, 0)

[CONS] <- ( MSG 1, 1)
[CONS] <- ( MSG 3, 1)
[CONS] <- ( MSG 5, 1)
[CONS] <- ( MSG 7, 1)
[CONS] <- ( MSG 9, 1)
[CONS] <- ( MSG 2, 0)
[CONS] <- ( MSG 4, 0)
[CONS] <- ( MSG 6, 0)
[CONS] <- ( MSG 8, 0)
[CONS] <- ( MSG 10, 0)
```

Ilustración 3 - Ejecución del Ejercicio 3

En la imagen se aprecia cómo el hilo productor envía, en este caso, 10 mensajes numerados del 1 al 10 en orden ascendente. Los mensajes con identificador par tienen prioridad 0, y los impares, prioridad 1.

A medida que el hilo consumidor obtiene los mensajes de la cola, se advierte que no los recibe en el mismo orden en que fueron enviados. Esto se debe a la prioridad de cada mensaje. Los mensajes con prioridad más alta se adelantan en la cola. Los 5 mensajes con prioridad 1 se recibieron antes que los mensajes con prioridad 0, a pesar de que algunos fueron enviados más tarde. Dentro de cada prioridad sí que impera el orden de llegada.

Por lo tanto, queda demostrado que el comportamiento de las colas de mensajes de POSIX está basado en prioridades y, dentro de cada grupo de prioridades, en FIFO.

EJERCICIO 4 – Memoria Compartida

Especificación

- 4.1. Cree un programa que utilice memoria compartida en POSIX (`shm_open()`, `mmap()`) para compartir datos entre dos procesos.
- 4.2. Implemente alguna medida de protección de la memoria compartida, como por ejemplo **semáforos**.
- 4.3. Documente lo que ha realizado.

Diseño

Para hacer este ejercicio se ha propuesto una estructura de productor y consumidor. En este caso, se han usado dos códigos fuente, que constituyen dos procesos distintos.

La interacción entre ambos procesos es la siguiente: el productor escribe un mensaje de texto letra a letra en la zona de memoria compartida, y el consumidor lee de ahí de la misma forma. La memoria compartida entre los procesos tiene el tamaño para guardar un carácter, de modo que el productor escribe, el consumidor lee, y el ciclo se repite hasta que el mensaje haya sido procesado.

Para evitar condiciones de carrera, se han usado dos semáforos distintos, uno para el productor y otro para el consumidor. Además, estas estructuras de control permitirán que los procesos se coordinen para acceder a la memoria compartida.

Implementación

El proceso productor se encarga de crear un objeto de memoria compartida (`shm_open()`), lo dimensionarlo (`ftruncate()`) y finalmente mapear ese objeto a su espacio de direcciones (`mmap()`). El proceso consumidor solo abre el recurso compartido existente y mapea su dirección.

Para garantizar que el productor siempre escribe en la memoria compartida antes de que el consumidor lea de la misma, y que ambos procesos se alternan se ha usado la siguiente configuración y esquema de semáforos.

```

/* Productor */

sem_prod = sem_open(1);
sem_cons = sem_open(0);

for (i=0; i<strlen(msg)+1; i++) {
    sem_wait(sem_prod);
    *shmem = msg[i];
    sem_post(sem_cons);
}

```

```

/* Consumidor */

endstr = 0;
for (i=0; !endstr; i++) {
    sem_wait(sem_cons);
    msg[i] = *shmem;
    if ( *shmem == '\0' )
        endstr = 1;
    sem_post(sem_prod);
}

```

Cada proceso hace un `sem_wait()` a su semáforo antes de usar la memoria compartida. Si esta llamada causa un bloqueo del proceso, el otro lo desbloqueará con `sem_post()` cuando ya haya usado la memoria compartida. De esta forma los procesos se bloquean y desbloquean entre ellos, permitiendo así un acceso coordinado al recurso compartido y un correcto procesamiento del mensaje carácter a carácter.

Ejecución

A continuación, se muestra una captura de pantalla de la ejecución de los procesos productor y consumidor. El productor muestra el mensaje a enviar a través de la memoria compartida. El consumidor procesa el mensaje y lo muestra.

<pre> milax@casa:~/STR/str-p2/src/ex4\$./prod.out Mensaje: Un dia de partit Al Gol Nord vaig anar Només entrar a la grada Em vaig enamorar Escribiendo en memoria compartida... Mensaje Enviado milax@casa:~/STR/str-p2/src/ex4\$ </pre>	<pre> milax@casa:~/STR/str-p2/src/ex4\$./cons.out Reciviendo de memoria compartida... Mensaje Recibido: Un dia de partit Al Gol Nord vaig anar Només entrar a la grada Em vaig enamorar milax@casa:~/STR/str-p2/src/ex4\$ </pre>
--	--

Ilustración 4 - Ejecución del Ejercicio 4

NOTA: Para la correcta ejecución de los procesos, se debe ejecutar el productor antes que el consumidor.

EJERCICIO 5 – Señales de Tiempo Real

Especificación

Desarrollar un programa en C que demuestre el uso de Señales de Tiempo Real POSIX (SIGRTMIN a SIGRTMAX) cumpliendo los siguientes requisitos:

- 5.1. Utilizar `sigqueue()` para enviar una señal que transporte un dato adjunto (un número entero).
- 5.2. Implementar un manejador de señales (*handler*) capaz de interceptar dicha señal y extraer el dato enviado.

Diseño

Configuración del Receptor: Se utiliza la estructura `sigaction` con el flag `SA_SIGINFO`. Esto instruye al Sistema Operativo para que no solo interrumpa el proceso, sino que también entregue una estructura `siginfo_t` con metadatos al *handler*.

Envío: En lugar de la función estándar `kill`, se usa `sigqueue`. El dato (entero) se encapsula en una union `sigval`, la cual el Kernel copia y transporta hasta el manejador.

La unión se comporta como un `struct` pero los miembros se apilan uno sobre el otro. Es decir, si se escribe un dato sobre un campo de la unión y a continuación se escribe otro dato sobre otro campo distinto (o el mismo) de la unión, se sobrescribe por el primero que había.

Recepción: El manejador se define con tres argumentos (`int`, `siginfo_t*`, `void*`) para poder acceder a la dirección de memoria donde el kernel depositó el dato.

Implementación

Estructuras de Datos:

- `union sigval`: Usada en el *main* para empaquetar el entero (`.sival_int`).
- `siginfo_t`: Usada en el *handler* para desempaquetar el entero (`si->si_value.sival_int`).

Funciones Clave:

- `sigaction()`: Vincula SIGRTMIN con la función `rt_handler`.
- `sigqueue(pid, signal, value)`: Envía la señal y el valor al propio proceso (`getpid()`).

Ejecución

```
milax@casa:~/STR/str-p2/src/ex5$ make run
gcc ex5.c -o ex5.out -lc
ex5.c: In function 'rt_handler':
ex5.c:15:15: warning: comparison between pointer and integer
   15 |         if (value != NULL){
       |                ^~
./ex5.out
Sending SIGRTMIN with value 42
Caught signal 34. Data recieved: 42
Sending SIGRTMIN with value 100
Caught signal 34. Data recieved: 100
Sending SIGRTMIN with NO value 0
Caught signal 34. NO DATA recieved
Exiting after success
```

Ilustración 5 - Ejecución del Ejercicio 5

EJERCICIO 6 – Inversión de Prioridades

Especificación

- 6.1. Desarrollar un escenario donde pueda haber una inversión de prioridades.
- 6.2. Usar POSIX mutex con herencia de prioridades para "bloquear" la inversión de prioridades.
- 6.3. Probar y documentar el comportamiento previo y posterior a la inversión de prioridades.

Diseño

El objetivo del diseño es forzar una condición de carrera conocida como Inversión de Prioridad, donde una tarea crítica (Alta Prioridad) es retrasada indefinidamente por una tarea de importancia menor (Media Prioridad).

Para lograrlo, se diseña un sistema con tres actores concurrentes que compiten por la CPU y un recurso compartido (mutex):

1. **Hilo Low (Prio 10):** Adquiere el mutex primero y ejecuta una tarea larga.
2. **Hilo Medium (Prio 20):** No necesita el mutex, pero consume CPU intensivamente. Su función es interrumpir (expropiar) al hilo Low.
3. **Hilo High (Prio 30):** Intenta adquirir el mutex (ya ocupado por Low) y se queda bloqueado esperando.

Implementación

Se ha forzado al planificador de Linux a usar la política de planificación SCHED_FIFO porque es una política de tiempo real estricta. A diferencia de la planificación estándar de Linux (SCHED_OTHER), SCHED_FIFO no aplica *time-slicing* (turnos de tiempo equitativos). Si un hilo de mayor prioridad (Medium) quiere ejecutarse, el sistema operativo garantiza que expulsará al hilo de menor prioridad (Low) y no le devolverá la CPU hasta que Medium termine. Sin SCHED_FIFO, el planificador podría ser "justo" y dejar correr a Low de vez en cuando, rompiendo el experimento de inversión.

- **Afinidad de CPU:** Se utiliza `sched_setaffinity` y `CPU_SET(0, ...)` al inicio del *main*. Esto obliga al proceso y a todos sus hilos a ejecutarse estrictamente en el Núcleo 0.
- **Justificación:** Sin esto, en un sistema moderno *multicore*, los hilos Low y Medium correrían en paralelo en núcleos distintos, evitando el bloqueo y haciendo imposible reproducir la inversión de prioridad.

- **Gestión de Prioridades:** Se utiliza `pthread_setschedparam()` para asignar prioridades estáticas ascendentes (10, 20, 30) bajo la política `SCHED_FIFO`.
- **Mecanismo de Solución (Herencia de Prioridad):** El código utiliza una directiva de preprocesador (`#ifdef USE_PRIO_INHERITANCE`) para compilar dos versiones del comportamiento del mutex:
 1. **Sin Herencia:** El mutex es estándar. Se produce la inversión (High espera a Low, Low es bloqueado por Medium).
 2. **Con Herencia** (`PTHREAD_PRIO_INHERIT`): Se configura el atributo del mutex con `pthread_mutexattr_setprotocol`. Cuando High se bloquea esperando a Low, el kernel eleva temporalmente la prioridad de Low hasta el nivel de High (30), permitiéndole terminar su tarea sin ser interrumpido por Medium.

Ejecución

- Ejecución con inversión de prioridades. No se usa `pthread_mutexattr_setprotocol()` para evitar la condición de carrera:

```
milax@casa:~/STR/str-p2/src/ex6$ sudo make run
./ex6.out
--- CPU Affinity set to CORE 0 (Simulating Single Core) ---
NO PRIORITY INHERITANCE (INVERSION WILL HAPPEN)LOW: Trying to lock mutex
LOW: Locked mutex
MED: Running (I don't need the lock, I just burn CPU)...
HIGH: Trying to lock mutex
MED: Finished
LOW: unlocking mutex
HIGH: LOCKED mutex Finally!. Critical section.
HIGH: Finished
```

Ilustración 6 - Ejecución del Ejercicio 6 (Con Inversión de Prioridades)

- Ejecución sin inversión de prioridades. Usando `pthread_mutexattr_setprotocol()` para evitar la condición de carrera:

```
milax@casa:~/STR/str-p2/src/ex6$ sudo make run
gcc ex6.c -o ex6.out -lc
./ex6.out
--- CPU Affinity set to CORE 0 (Simulating Single Core) ---
ENABELING PRIORITY INHERITANCE (INVERSION WON'T HAPPEN)
LOW: Trying to lock mutex
LOW: Locked mutex
MED: Running (I don't need the lock, I just burn CPU)...
HIGH: Trying to lock mutex
LOW: unlocking mutex
HIGH: LOCKED mutex Finally!. Critical section.
HIGH: Finished
MED: Finished
```

Ilustración 7 - Ejecución del Ejercicio 6 (Sin Inversión de Prioridades)